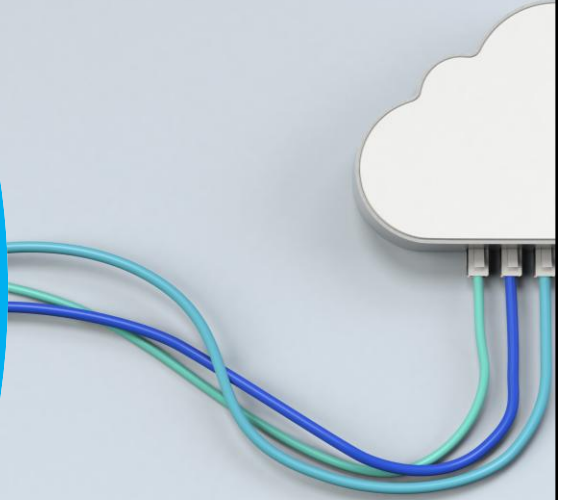


Darstellungsschicht

NETZWERKTECHNIK / SEMESTER 1 UND 2



1

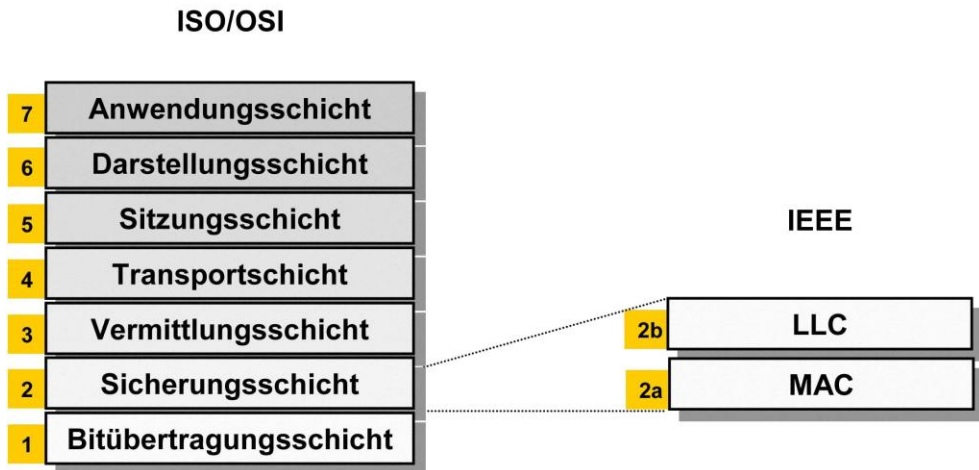
AGENDA

- 01 DARSTELLUNGSSCHICHT
- 02 ZEICHENSÄTZE UND KODIERUNG
- 03 DATENKOMPRESSION

2

NETZWERKTECHNIK / SEMESTER 1 und 2

Einordnung im ISO/OSI Modell



[Quelle: https://www.airnet.de/cr1-gfe/de/html/TrFddIIEEE802_learningObject6.xml] -
 letzter Abruf 21.07.2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

3

3

01

Darstellungsschicht

4

Darstellungsschicht

Die Aufgabe der **Darstellungsschicht (engl. Presentation Layer)** ist es, den Kommunikationspartnern eine einheitliche Interpretation der Daten zu ermöglichen, d. h. Daten in einem einheitlichen Format zu übertragen.

Der Darstellungsschicht sind grundsätzlich folgende Aufgaben zugeordnet:

- die Darstellung der Daten (Syntax),
- die Datenstrukturen zur Übertragung der Daten
- die Darstellung der Aktionen an diesen Datenstrukturen, sowie
- Datentransformationen.

Anwendungen sollen syntaxunabhängig miteinander kommunizieren können.

Anwendungsspezifische Syntax kann von der Darstellungsschicht in eine einheitliche Form umgewandelt und dann übertragen werden.

Hinweis: Die Darstellung auf Schicht 6 muss nicht der Darstellung auf Schicht 7 (Anwendungsschicht) entsprechen. **Die Darstellungsschicht ist für die Syntax der Nutzdaten verantwortlich**, die Semantik verbleibt bei den Anwendungen.

Unter **Syntax** versteht man die Darstellung von Daten nach bestimmten Regeln (Grammatik). Wird Daten eine Bedeutung zugeordnet, spricht man von Information (Aufgabe der **Semantik**).

Aufgaben der Darstellungsschicht

Den grundlegenden Aufgaben der Darstellungsschicht lassen sich konkrete Funktionen zuordnen:

- **Kodierung der Daten**
 - Übersetzung zwischen Zeichensätzen und Codewörtern gemäß standardisierter Kodierungsvorschriften
 - Kompression von Daten vor dem Senden (Entfernung von unerwünschter Redundanz)
 - Verschlüsselung
- **Strukturierte Darstellung** von Daten
 - plattformunabhängige, einheitliche Darstellung
 - Übersetzung zwischen verschiedenen Datenformaten
 - Serialisierung strukturierter Daten für die Übertragung

Wie auch bei der Sitzungsschicht gilt: Der Darstellungsschicht zuordenbare Protokolle erfüllen häufig auch Funktionen anderer Schichten.

Beispiel TLS: Die Verschlüsselungsfunktionen von TLS können der Darstellungsschicht zugeordnet werden. Funktionen wie Verbindungsaufbau und -abbau sowie Authentifizierung und Autorisierung fallen hingegen in den Bereich der Sitzungsschicht.

02

Zeichensätze und Kodierung

7

NETZWERKTECHNIK / SEMESTER 1 und 2

Zeichensätze und Kodierung

Daten liegen in einer von zwei Formen vor:

Textzeichen bzw. **Symbole** in lesbarer Form („human readable“), z. B. Buchstaben, Textrepräsentation von Zahlen, Sonderzeichen. . .

- Ein **Zeichensatz** ist eine Menge textuell darstellbarer Zeichen sowie deren Zuordnung zu einem **Codepoint**.
- Wie die Codepoints eines bestimmten Zeichensatzes in binärer Form (also mittels einer Sequenz von Bits) dargestellt werden wird durch **Kodierungsvorschriften** festgelegt.
- Für einen Zeichensatz können ggf. mehrere mögliche Kodierungen existieren.

Binäre Daten (also eine Sequenz von Bits), z. B. binäre Darstellungen von Buchstaben, Zahlen und Symbolen aber auch Bilder, Musik, Filme, etc. in digitaler Form. . .

- Ein **Datum** ist eine für Computer verarbeitbare „Einheit“, eine kurze Sequenz von Bits, deren Länge meist ein Vielfaches von 8 bit ist.
- Was ein Datum repräsentiert – eine Zahl, ein Zeichen, einen Teil davon oder doch ein Bild – ist kontextabhängig
- Ist bekannt, dass es sich bei den vorliegenden Daten um Text handelt, welche Kodierung verwendet wurde und um welchen Zeichensatz es sich handelt, lassen sich die binären Daten leicht wieder in Textzeichen übersetzen.

8

Zeichensätze und Kodierung

- Ein **Zeichensatz** besteht aus einer Menge von Zeichen, bei der jedes **Zeichen** mit einem **Codepoint** verknüpft ist.
- Beispiele für Zeichensätze sind **ASCII**, **ISO-8859-15 (ISO-8859-1 mit C)** und **Unicode**.
- Ein Zeichensatz kann **druckbare Zeichen** sowie **Steuerzeichen** enthalten.
 - ASCII definiert 128 Zeichen.
 - Davon sind die Zeichen 0 – 31 und 127 **Steuerzeichen**.
 - Beispiele für Steuerzeichen sind Zeilenumbruch, Tabulator und Protokollzeichen.
 - Ursprünglich wurden darüber Drucker und Terminals angesteuert.
- **ISO-8859-15 definiert 256 Codepoints.**
 - Dabei entsprechen die Codepoints 0 – 127 dem ASCII Zeichensatz.
 - Weitere Codepoints sind mit Sonderzeichen europäischer Sprachen sowie dem Eurozeichen verknüpft.
 - ISO-8859-15 war vor Unicode ein im westeuropäischen Sprachraum weit verbreiteter Zeichensatz.

Zeichensätze und Kodierung

- **Unicode**
 - Unicode ist ein Zeichensatz, der in Version 13.0 143 859 Zeichen definiert.
 - Die Größe des Coderaums (Anzahl möglicher Zeichen) beträgt bei Unicode 1 114 112 Zeichen, gegliedert in 17 Ebenen mit je 216 Zeichen.
 - Unicode hat das Ziel, alle Schriftkulturen und Zeichensysteme abzubilden.
 - Die Codepoints 0 – 255 entsprechen ISO-8859-1.
 - Im Gegensatz zu eingeschränkten Zeichensätzen wie z. B. ASCII und ISO-8859-15 wird der Zeichensatz bei Unicode regelmäßig aktualisiert und erweitert.
 - Es sind mehrere Zeichencodierungen definiert, die Codepoints für alle Unicode-Zeichen beinhalten, mit denen die Codepoints auf konkrete binäre Darstellungen abgebildet werden, u. a. UTF-32LE (Little Endian), UTF-32BE (Big Endian), UTF-16LE und UTF-16BE und UTF-8.

Kodierung

- Zur Übertragung müssen Zeichen (bzw. die entsprechenden Codepoints) **kodiert** werden. Man unterscheidet zwischen
 - Fixed-Length Codes**, bei welchen alle Zeichen mit Codewörtern derselben Länge kodiert werden, z. B. ASCII (7 bit) oder UCS-2 (16 bit) und
 - Variable-Length Codes**, bei denen Zeichen mit Codewörtern unterschiedlicher Länge kodiert werden, z. B. UTF-8 (1 – 4 B).

Die möglichen Kodierungsverfahren hängen dabei vom jeweiligen Zeichensatz ab:

- ASCII und ISO-8859-15 definieren die Kodierung gemeinsam mit dem Zeichensatz.
 - Codewörter sind bei ASCII 7 bit lang, wobei das highest-order Bit eines Oktetts stets 0 ist.
 - ISO-8859-15 verwendet 8 bit lange Codewörter.
- Unicode definiert keine Kodierung sondern nur einen Zeichensatz. Häufig wird zur Kodierung von Unicode **UTF-8** verwendet, welches (mit Einschränkungen) kompatibel zu ASCII ist.

Unicode Transformation Format (UTF-8)

Unicode-Bereich	Länge	binäre UTF-8 Kodierung	kodierbare Bits
U+0000 – U+007F	1 B	0xxxxxxx	7
U+0080 – U+07FF	2 B	110xxxxx 10xxxxxx	11
U+0800 – U+FFFF	3 B	1110xxxx 10xxxxxx 10xxxxxx	16
U+10000 – U+1FFFFF	4 B	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	21

UTF-8 kodiert den Unicode Zeichensatz abhängig vom Codepoint mit 1 – 4 B langen Codewörtern:

- Die Darstellung U+xxxx ist lediglich eine Notation der Codepoints für Unicode. Die hexadezimalen Ziffern geben dabei den Wert der **kodierten Bits** eines Codeworts an.
- Bei Codewörtern länger als 1 B gibt die Anzahl der führenden 1-en vor der ersten 0 im ersten Oktett die Länge des Codeworts an.
- Die beiden highest-order Bits aller nachfolgenden Oktette eines Codeworts sind 10.
- Bei Codewörtern, die nur aus einem Oktett bestehen, ist das highest-order Bit stets 0 (vgl. ASCII).

Eigenschaften:

- UTF-8 ist rückwärts-kompatibel zu ASCII: ASCII kodierter Text ist valides UTF-8 und kann dementsprechend ohne Konvertierung als Unicode interpretiert werden.
- UTF-8 ist präfixfrei und selbstsynchronisierend.
- Nicht alle Kombinationen sind gültige Codewörter (Kompatibilitätsgründe mit UTF-16).

Beispiele

Beispiel 1: Kodierung des Umlauts ä

- **Unicode:** Codepoint 228, Codewort in UTF-8: U+00E4 = 11000011 10100100
- („LATIN SMALL LETTER A WITH DIAERESIS“)
- **ISO-8859-1 und ISO8859-15:** Codepoint 228, Codewort: 11100100
Obwohl beide Zeichensätze eine Teilmenge von Unicode sind, ergeben sich bei Unicode bei der Zeichencodierung andere Codewörter.
- **ASCII:** Keine Kodierung möglich, da Umlaute nicht Teil des Zeichensatzes ist.

Beispiel 2: Kodierung des Eurosymbols €

- **Unicode:** Codepoint 8364, Codewort in UTF-8: U+20AC = 11100010 10000010 10101100
- **ISO-8859-15:** Codepoint 164, Codewort 10100100
- **ISO-8859-1 und ASCII:** Keine Kodierung möglich

Kodierung

Vom verwendeten Zeichensatz nicht unterstützte Zeichen können häufig mittels **Zeichen-Entität-Referenzen** kodiert werden:

- **XML** erlaubt beispielsweise die Kodierung beliebiger Unicode-Zeichen durch Angabe des Codepoints, z. B. ä
- **HTML** erlaubt die Kodierung häufiger verwendeter Zeichen mittels **named entities**, z. B. ä
- **LATEX** erlaubt die Kodierung verschiedener Zeichen auf ähnliche Art, z. B. \"a1

Die Syntax ist aber abhängig vom jeweils verwendeten Protokoll (HTTP, SMTP → später) bzw. der Anwendung.

Strukturierte Darstellung

Damit Anwendungen Daten austauschen können, müssen diese eine einheitliche Syntax für die ausgetauschten Daten verwenden. Möglichkeiten hierfür sind:

- **(gepackte) structs / serialisierte Speicherbereiche**
 - Daten werden so wie sie im Speicher vorliegen übertragen. Integration zwischen verschiedenen Systemen schwierig, weil diese dieselben Datenstrukturen (und Compiler) verwenden müssen. Erweiterungen/Änderungen sind nur dann möglich, wenn alle beteiligten Systeme gleichzeitig aktualisiert werden.
- **Ad-hoc Datenformate**
 - Datenformat wird bei Bedarf „entworfen“. Problematisch sind hierbei die Dokumentation, Eindeutigkeit, Fehlerfreiheit (wie gut und sicher kann das Format geparsed werden) und Erweiterbarkeit.
- **Strukturierte Serialisierungsformate** wie XML oder JSON.

Beispiel: JavaScript Object Notation (JSON)

- Definiert in ECMA-404 [1] und RFC 7159 [4]: Ursprünglich von JavaScript abgeleitet, mittlerweile sprachenunabhängiges Format.
- Daten werden struktuiert und in lesbarer („human-readable“) Form als Text übertragen.

JSON

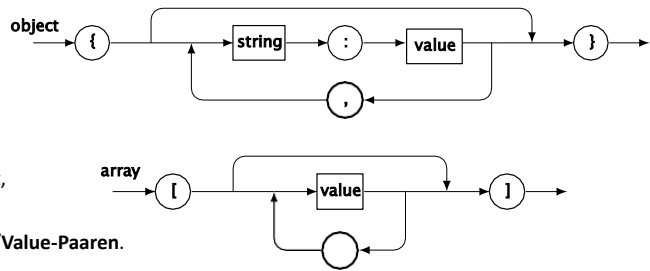
JSON definiert die folgenden Datentypen: number, string, boolean, array, object, null

- Zwischen den Elementen kann (beliebiger) Whitespace eingefügt werden.
- JSON wird im allgemeinen als UTF-8 kodiert.

```
{
  "Image": {
    "Width": 800 ,
    "Height": 600 ,
    "Title": " View from 15 th Floor",
    "Thumbnail": {
      "Url": " http :// www .example .com /image /481989943 ",
      "Height": 125 ,
      "Width": 100
    },
    "Animated" : false ,
    "IDs" : [116 , 943 , 234 , 38793]
  }
}
```

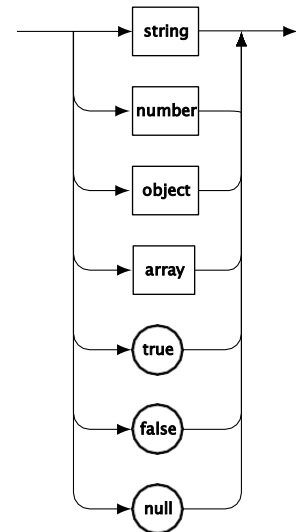

JSON Objects und Arrays

- JSON-Syntax, in RFC4627 über Augmented BNF spezifiziert, kann auch über Syntax-Diagramm spezifiziert werden.
- Ein JSON-Objekt ist eine ungeordnete Sammlung von **Key/Value-Paaren**.
- Der Key (Schlüssel) ist ein Unicode-String.
- Die Values (Werte) können unterschiedliche Typen aufweisen.
- Eine Sammlung von JSON-Objekten ist konzeptuell vergleichbar mit Hashmaps bzw. Dictionaries verschiedener Programmiersprachen.
- Arrays sind (geordnete) Listen (leere Listen sind erlaubt).
- Bei der Übertragung bleibt die Reihenfolge der Elemente innerhalb einer Liste erhalten.
- Listen können wiederum Werte von unterschiedlichen Typen aufweisen.



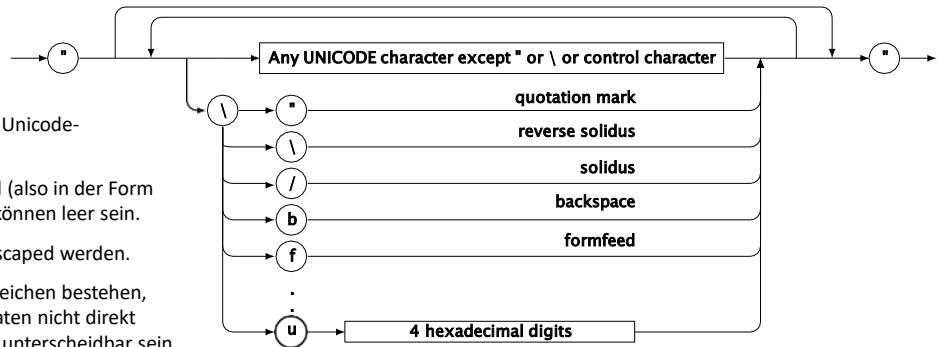
JSON Values

- Werte (values) können in JSON selbst wieder beliebige Typen sein.
- Booleans werden durch true und false repräsentiert.
- null entspricht dem Nullpointer (null, None, nil, NUL, etc.) der verschiedenen Sprachen.



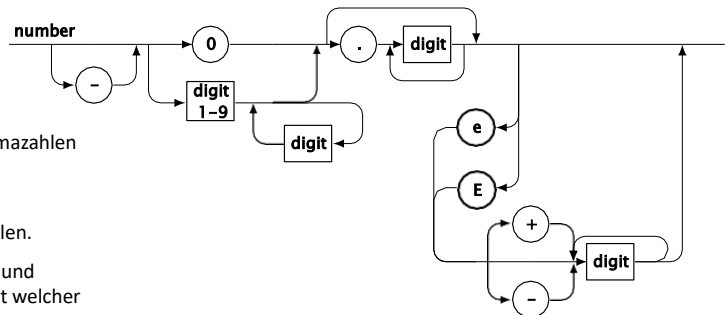
JSON Strings

- Strings können beliebige Unicode-Zeichenketten sein.
- Strings werden gequoted (also in der Form "<text>" notiert) und können leer sein.
- Steuerzeichen müssen escaped werden.
- Da Strings aus Unicode-Zeichen bestehen, können beliebige Binärdaten nicht direkt kodiert werden (es muss unterscheidbar sein, ob es sich um einen String oder um Binärdaten handelt).
- Das Problem kann durch Serialisierung der Binärdaten z. B. mittels Base64 umgangen werden, oder mittels BSON (Binary JSON).



JSON Numbers

- Numbers sind dezimal notierte Zahlen
- Es wird nicht zwischen Ganzzahlen und Gleitkommazahlen unterschieden.
- Hintergrund hierfür: JavaScript – worauf JSON ja ursprünglich basierte – kennt nur Gleitkommazahlen.
- Es ist implementierungsabhängig, ob Ganzzahlen und Gleitkommazahlen unterschieden werden und mit welcher Präzision diese abgebildet werden.
- Implementierungsabhängig ist auch, ob die Zahlenwerte 42, 4.2e1 und 42.0 als gleich angesehen werden.



03

Datenkompression

21

21

NETZWERKTECHNIK / SEMESTER 1 und 2

Datenkompression

Bei Kompressionsverfahren muss unterscheiden werden:

1. Verlustfreie Komprimierung (engl. lossless compression)

- Komprimierte Daten können verlustfrei, d. h. exakt und ohne Informationsverlust, wiederhergestellt werden.
- Verlustfrei komprimierte Dateiformate sind beispielsweise ZIP, PNG1 (Bilder), FLAC2 (Musik), . . .

2. Verlustbehaftete Komprimierung (engl. lossy compression):

- Komprimierte Daten können im Allgemeinen nicht wieder exakt rekonstruiert werden.
- Es tritt also ein Verlust von Information bei der Komprimierung auf.
- Dafür ermöglichen diese Verfahren meist höhere und in Abhängigkeit des Verlustfaktors variable Kompressionsraten.
- Verlustbehaftet komprimierte Dateiformate sind beispielsweise MP3, MPEG, JPEG, . . .



[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

22

22

Huffman-Code

Viele Protokolle komprimieren Daten vor dem Senden (Quellenkodierung).

Häufig nutzen Dateiformate den Huffman-Code als Backend bzw. in Kombination mit weiteren Verfahren, z. B. ZIP / DEFLATE, JPEG.

Grundlegende Idee der Huffman-Kodierung:

- Nicht alle Textzeichen treten mit derselben Häufigkeit auf, z. B. tritt der Buchstabe „E“ in der deutschen Sprache mit einer Häufigkeit von 17,4 % gefolgt von „N“ mit 9,8 % auf.
- Anstelle Zeichen mit uniformer Codewortlänge zu kodieren (z. B. ASCII-Code), werden häufigen Zeichen kürzere Codewörter zugewiesen.
- Die Abbildung zwischen Zeichen und Codewörtern bleibt dabei eindeutig und umkehrbar, weswegen es sich um ein verlustloses Kompressionsverfahren handelt.
- [Beispiel: Huffman-Code - lernen mit Serlo!](#)

Huffman-Code erzeugen

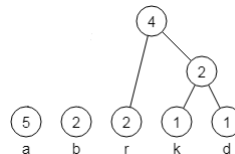
Du möchtest zu dem Text abrakadabra den Huffman-Baum konstruieren.



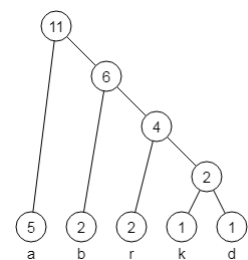
In Schritt 1 erzeugte Knoten



Der erste in Schritt 2 neu erzeugte Knoten



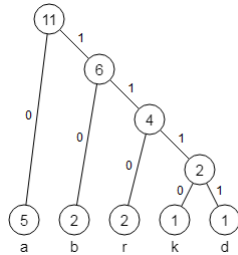
Der nächste in Schritt 2 neu erzeugte Knoten



Der fertige Huffman-Baum

Huffman-Code erzeugen

Die Kodierung des gesamten Textes abrakadabra lautet somit: **01011001110011110101100**. Die Länge der Kodierung beträgt 23 Bit.



Du markierst nun die Kanten des Baumes mit 0 und 1, wie im Bild zu sehen.

Wenn du dann die Wege von der Wurzel zu den Blättern verfolgst, erhältst du die **Codewörter des Huffman-Codes**, also **0** für **a**, **10** für **b**, **110** für **r**, **1110** für **k** und **1111** für **d**.

Huffman-Code wieder dekodieren

Wenn du diese Folge von Nullen und Einsen wieder dekodierst, woher weißt du dann, wo die Kodierung eines Buchstabens zu Ende ist und wo die nächste anfängt?

- Hier kommt die Fano-Bedingung ins Spiel: **Kein Codewort ist Anfangswort eines anderen Codewortes!** Wenn du also die erste Ziffer 0 liest, hast du den Buchstaben a schon erkannt! Und mit den folgenden Ziffern 10 das b!
- Praktisch dekodierst du die Bitfolge 01011001110011110101100, indem du den Huffman-Baum von der Wurzel aus jeweils abhängig vom nächsten Bit durchläufst. Wenn du ein Blatt erreichst, gibst du den zugehörigen Buchstaben aus.
- Mit dem ersten Bit 0 der Bitfolge landest du gleich bei einem Blatt und gibst das a aus.
- Anschließend startest du wieder bei der Wurzel. Mit den folgenden Bits 10 erreichst du wieder ein Blatt und gibst das b aus.
- Dann startest du wieder bei der Wurzel und erreichst mit den folgenden Bits 110 das Blatt mit dem Buchstaben r.
- Und so weiter...